

Checklist

Platform Creation

1. Repository and Local Tooling

- Create or clone the `platform-core` repository.
- Create the base repository structure:
 - `.circleci/`
 - `modules/`
 - `scripts/`
 - `tests/`
- Configure the repository as a Python Pulumi project.
- Install and verify Docker.
- Install and verify `kubect1`.
- Install and verify Kind.
- Install and authenticate the Pulumi CLI.
- Install Helm.
- Install BATS for infrastructure testing.
- Confirm all required commands run successfully from the development machine.

2. Platform Environments

- Create the `platform-sandbox` Pulumi stack.
- Create the `app-dev` Pulumi stack.
- Create the `app-prod` Pulumi stack.
- Create a separate Pulumi YAML configuration file for each stack.
- Define a unique cluster name for each environment.
- Pin the Kubernetes/Kind node image version.
- Define deployment readiness/wait settings.
- Keep security and governance settings consistent across environments.
- Use smaller capacity where appropriate for non-production environments.
- Confirm platform environments and application environments are treated as separate SDLC concepts.

3. Network Foundation

- Create `modules/network.py`.
- Define a `PortMap` configuration model.
- Define a `NetworkConfig` configuration model.
- Assign a unique Docker network to every platform environment.
- Assign non-overlapping CIDR ranges.
- Check CIDRs against VPN and host-network ranges.
- Define the cluster virtual network range.
- Define pod CIDRs.
- Define service CIDRs.
- Configure required ingress port mappings.
- Map HTTP traffic, such as host port `8080`.
- Map HTTPS traffic, such as host port `8443`.
- Implement Docker-network creation through Pulumi.
- Generate the Kind cluster configuration programmatically.
- Bind the Kind cluster to the appropriate Docker network.
- Apply a zero-trust approach by minimizing unnecessary exposure.
- Verify CIDRs do not overlap between environments.

4. Kubernetes Runtime

- Create `modules/cluster.py`.
- Define a `ClusterConfig` configuration model.
- Configure the cluster name.
- Configure the pinned Kind image.
- Configure cluster readiness timeout values.
- Implement Kind cluster creation as a Pulumi-managed command.
- Ensure network creation completes before cluster creation.
- Generate the cluster kubeconfig.
- Configure the Pulumi Kubernetes provider with the generated kubeconfig.
- Enable server-side apply where appropriate.
- Export useful Pulumi outputs such as:
 - cluster name
 - Docker network
 - kubeconfig
- Deploy `platform-sandbox` with `pulumi up`.
- Confirm `kubectl` can connect to the cluster.
- Confirm all Kubernetes nodes reach `Ready`.

5. Infrastructure Validation Tests

- Create `tests/infrastructure.bats`.
- Add test setup that retrieves Pulumi stack outputs.
- Configure `KUBECONFIG` automatically during tests.
- Capture the Docker network name.
- Capture the cluster name.
- Add teardown logic to remove temporary kubeconfig files.
- Test that the expected Docker network exists.
- Test that the Kubernetes API is reachable.
- Test that at least one Kubernetes node exists.
- Test that required nodes are ready.
- Run the infrastructure test suite locally.
- Confirm tests fail clearly when infrastructure is unavailable.

6. Code Quality and Security Checks

- Add Python formatting checks with Black.
- Add type checking with mypy.
- Add import-order checking with isort.
- Add static-analysis checks.
- Add security scanning.
- Consider an open-source SAST tool such as Semgrep.
- Consider Trivy for container, dependency, IaC, and SBOM scanning.
- Configure checks to fail the pipeline on significant violations.
- Tune scanning rules to avoid excessive false positives.

7. CI/CD Pipeline

- Create `.circleci/config.yml`.
- Configure the required machine executor.
- Configure access to the local/platform deployment environment.
- Add a lint stage.
- Add pre-deployment testing.
- Add a Pulumi preview stage.
- Add a Pulumi deployment stage.
- Add post-deployment validation.
- Run sandbox deployments from commits to `main`.
- Restrict regular branch deployments to the main branch.

- Use Git tags for production-oriented releases.
- Deploy and validate `platform-sandbox` before application environments.
- Deploy `app-dev` after successful sandbox validation.
- Add a manual approval gate before `app-prod`.
- Deploy `app-prod` only after approval.
- Run validation tests after every environment deployment.
- Keep approval gates only where they provide meaningful governance value.

8. GitOps Foundation

- Create a separate `platform-gitops` repository.
- Keep infrastructure provisioning in `platform-core`.
- Keep runtime/application configuration in `platform-gitops`.
- Avoid managing application Kubernetes manifests directly with IaC.
- Use Helm and/or Kustomize for Kubernetes application lifecycle management.
- Install Flux in the platform cluster.
- Create a `flux-system` namespace.
- Create the Flux installation module in `platform-core`.
- Pin the Flux version.
- Verify Flux controllers become healthy.

9. App-of-Apps GitOps Structure

- Configure `platform-gitops` as the central coordination repository.
- Create environment-specific configuration directories.
- Add a configuration for `platform-sandbox`.
- Add configurations for application environments as needed.
- Define Flux `GitRepository` resources.
- Define Flux `Kustomization` resources.
- Point Flux to the correct repository path for each environment.
- Configure reconciliation intervals.
- Enable pruning where appropriate.
- Configure Flux to wait for resources to become healthy.
- Bootstrap Flux from `platform-core`.
- Verify Flux begins monitoring `platform-gitops`.
- Verify a Git commit triggers reconciliation.
- Confirm new team repositories can be onboarded declaratively.

10. Platform Services Repository Structure

- Create or identify the repository containing shared platform services.
- Add the repository to the GitOps configuration.
- Create environment-specific overlays/configurations.
- Use GitOps to deploy platform extensions rather than direct manual installation.
- Ensure platform-service versions are explicitly pinned.

11. Istio Service Mesh

- Add the Istio Helm repository as a GitOps source.
- Pin the Istio chart/version.
- Deploy the Istio base components.
- Deploy the Istio control plane.
- Configure namespace/service-mesh integration.
- Verify Istio components become healthy.
- Verify Envoy/data-plane integration where sidecars are enabled.
- Configure an Istio Gateway for controlled ingress.
- Configure `VirtualService` resources for traffic routing.
- Enable or enforce mTLS as required.
- Verify service-to-service encrypted communication.
- Add authorization/network policies where workloads require isolation.
- Test service routing through the gateway.
- Monitor the resource and latency overhead introduced by service-mesh proxies.

12. Observability Hooks

- Confirm Istio emits service metrics.
- Confirm Istio can produce distributed tracing information.
- Confirm logs and telemetry can be consumed by an observability system.
- Validate visibility into service-to-service communication.
- Validate that latency and traffic behavior can be inspected.
- Leave the full observability platform implementation for the dedicated observability stage/chapter if following the book sequence.

13. Policy-as-Code

- Add an Open Policy Agent/Rego policy directory.
- Install/configure `confTest` for pipeline policy checks.

- Define policies for invalid or unsafe configuration.
- Prevent container images from using floating `latest` tags.
- Require reproducible/pinned deployment versions.
- Reject wildcard Helm versions such as `v2.*.*`.
- Run policy checks before manifests reach the GitOps repository.
- Make policy violations fail the pipeline early.
- Add tests for both allowed and rejected policy cases.
- Keep policy enforcement consistent across environments.

14. Production Environment Gate

- Confirm sandbox deployment succeeds.
- Confirm sandbox validation tests succeed.
- Confirm `app-dev` deployment succeeds.
- Confirm `app-dev` validation succeeds.
- Require explicit approval for the production deployment.
- Record the approval through the CI/CD system.
- Deploy `app-prod`.
- Run production validation tests.
- Confirm security and governance controls match tested environments.

15. GitOps and Reconciliation Validation

- Make a controlled configuration change in Git.
- Confirm Flux detects the change.
- Confirm Flux applies the desired state.
- Confirm manual configuration drift is reconciled.
- Confirm failed reconciliations are visible.
- Confirm deleted Git-managed resources are handled according to the configured pruning policy.
- Verify Git remains the source of truth for runtime configuration.

16. Final Platform Validation

- All required Pulumi stacks exist.
- Every environment deploys successfully.
- Kubernetes clusters are accessible.
- Nodes are healthy.
- Network ranges do not conflict.

- CI linting passes.
- Pre-deployment tests pass.
- Infrastructure deployment tests pass.
- Security scans pass.
- Policy-as-Code checks pass.
- Flux is healthy.
- GitOps reconciliation works.
- Istio is healthy.
- Gateway routing works.
- mTLS works as intended.
- Required telemetry is available.
- Production approval gates work.
- Configuration drift can be detected/reconciled.

17. Release

- Commit all platform code and configuration.
- Push all repositories.
- Apply a semantic version tag, such as `v0.1.0`.
- Push the version tag.
- Confirm the tag triggers the expected release pipeline.
- Confirm the deployment completes across all target environments.
- Confirm every environment is deployed and configured.
- Record the working platform version as the baseline for future development.

Definition of Done

- Infrastructure is declarative and reproducible.
- Development teams receive a standardized Kubernetes runtime.
- Environment configuration is managed consistently.
- CI/CD automatically tests deployments.
- GitOps continuously reconciles runtime configuration.
- Platform services are deployed using Kubernetes-native tooling.
- Service communication is centrally secured and managed.
- Misconfigurations are blocked through Policy-as-Code.
- The platform can be rebuilt from source-controlled configuration.
- The platform is ready for the next stages of security, observability, and developer-experience development.

